Speech Command Recognition with Reinforcement Learning-Based Threshold Tuning

Kean Joshua N. Antonio
Holy Angel University
Lot 205 D1-J Anusension Street, Mabiga, Mabalacat City, Pampanga
(+63) 9957305224
antoniokeanjoshua@gmail.com

Josh Adriane G. Lim
Holy Angel University
Havana St. Mirus Res,
Mabiga, Mabalacat City, Pampanga
(+63) 9267801125
jglimhau@gmail.com

Glen Garcia Pasamonte
Holy Angel University
660 San Angelo Street, Lourdes Sur,
Angeles City Pampanga
(+63) 9761509739
glenpasamonte28@gmail.com

Amiel Miguel T. Vitug
Holy Angel University
36-20 Mt. Taal St. Talimundok, Dau,
Mabalacat City, Pampanga
(+63) 9662531657
atvitug1@gmail.com

1. PROBLEM STATEMENT

Voice User Interfaces (VUIs), particularly those used in IoT devices such as smart assistants, depend on speech command recognition to interpret user requests. Despite their usefulness, These systems are often affected by real-world factors such as background noise, accent variation, and differences in speaking style, which can reduce recognition accuracy. Misclassification of spoken commands may result in incorrect system responses, lower usability, and reduced user satisfaction. We propose the development of a Speech Command Recognition system that uses a Convolutional Neural Network (CNN) with spectrogram-based input features to classify short spoken commands. To further improve decision-making, Reinforcement Learning (RL) will be used to tune the classification threshold so that false positives and false negatives can be balanced according to asymmetric error costs.

2. SCOPE

The scope of this project will include audio command classification using a spectrogram-based CNN, reinforcement learning-based threshold tuning to optimize decision-making, evaluation using a public speech command dataset, and the development of an offline experimental prototype solely for research and academic purposes.

3. SUCCESS METRICS & CONSTRAINTS

The project will measure the practical performance of both the CNN-based classifier and the RL-based threshold optimization component. The CNN model is expected to achieve a minimum of 90% accuracy on the test set and a macro-F1 score of at least 0.88, ensuring credible and balanced classification across all speech command classes. Meanwhile, the RL component should achieve at least a 20% reduction in expected error cost relative to a fixed-threshold baseline, indicating a meaningful improvement in classification decision quality. As a practical constraint, the complete training and optimization process must remain computationally feasible and should finish within 90 minutes using an IDE and a single GPU with 4GB VRAM.

4. RELATED WORK

The Google Speech Commands dataset is a large collection for limited-vocabulary speech recognition, providing thousands of labeled audio recordings suitable for training deep learning models [1]. Convolutional Neural Networks have been applied to speech command recognition and improved through a hyperparameter selection method based on the Differential Evolution algorithm. The DE-based method achieved 91.5% accuracy, outperforming the Genetic Algorithm-based approach, which reached 87.7% [2]. More recent studies have shown that Mel-spectrogram-based CNN models remain effective for speech command recognition. A four-layer CNN trained on Mel spectrograms achieved 96.05% testing accuracy, with 95% micro-averaged and macro-averaged precision, recall, and F1-scores, demonstrating strong and balanced classification across command classes [3]. Transfer learning has also been applied to speech command detection, where an improved pretrained YAMNet model achieved 95.28% accuracy on the Speech Commands dataset, showing that pretrained audio models can also perform well in this task [4]. Reinforcement learning

has also been used in imbalanced classification, where a deep RL framework achieved more balanced performance than conventional methods and highlighted the importance of decision-threshold selection in classification outcomes [5]. Threshold estimation has also been studied in keyword spotting, where robust threshold selection was identified as a non-trivial task, and a score-calibration approach was proposed to simplify threshold selection and improve performance in noisy few-shot keyword spotting settings [6].

5. DATASET PLAN

The project will use the Google Speech Commands dataset, which is accessed through Kaggle. It contains audio recordings of spoken commands such as yes, no, up, down, left, and right, which are suitable for speech command classification. These audio clips are transformed into Mel-spectrogram-based representations for the CNN model. For splitting, the project follows the dataset's provided validation and testing lists, while all remaining usable samples are assigned to the training set. This follows the official partitioning of the dataset and helps reduce data leakage during evaluation.

6. PLANNED CNN/NLP/RL COMPONENTS AND MVP

The proposed system uses a Convolutional Neural Network (CNN) trained on Mel-spectrogram representations of audio signals to classify simple spoken commands. The CNN serves as the main deep learning model for recognizing short speech inputs based on spectrogram features. The Reinforcement Learning (RL) component is used as a threshold-tuning agent that adjusts the classifier's decision threshold to reduce expected error cost under asymmetric penalties for false positives and false negatives. For the project's Minimum Viable Product (MVP), the system implements a spectrogram-based CNN classifier together with an RL agent for decision-threshold optimization.

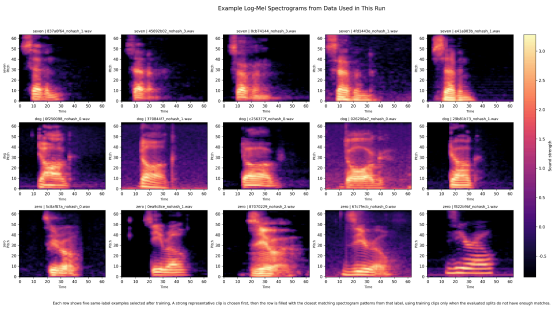
7. ETHICS STATEMENT

The Speech Command Recognition system carries three key ethical risks. First, accent and dialect bias arise because the Google Speech Commands v0.02 dataset skews toward American English speakers, potentially reducing accuracy for Filipino, British, or Indian accents; this is mitigated through audio augmentation and per-class performance documentation. Second, there is also a privacy concern because voice data can be sensitive. However, this project only uses a public dataset licensed under CC BY 4.0, does not collect new personal audio, and does not store additional participant recordings in the repository. Third, command misinterpretation, where commands like "no" may be classified as "go" is mitigated by RL-based threshold tuning and fallback handling for low-confidence predictions. No PII (Personal Identifiable Information) is present in the dataset; all speakers are fully anonymized with random IDs. This system is only an academic prototype and is not intended for public deployment or high-stakes use.

8. RESULTS

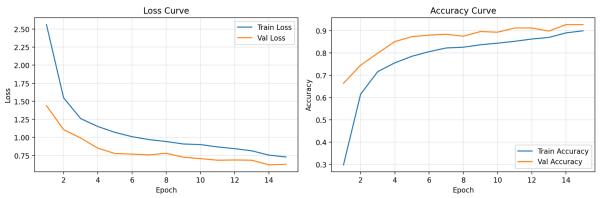
I. Main Output

Sample Log-Mel Spectrograms (medium_end)



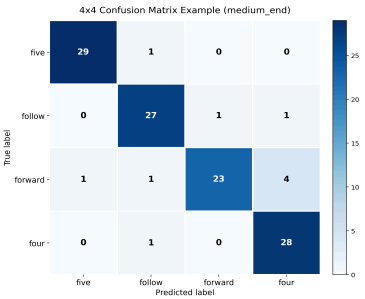
- Side-by-side examples of labeled Log-Mel spectrograms from the audio samples seven, dog, and zero

Training and Validation Accuracy/Loss Curves (medium_end)



- Training and validation loss and accuracy curves for the medium_end run across 15 epochs, showing steady convergence, improving accuracy, and reduced loss as training progressed.

Confusion Matrix (medium_end)



- The words "forward" and "four" show one of the most common misclassifications in the final test-set confusion matrix, being frequently mistaken with each other by the model.

Final Metrics Summary (medium_end)

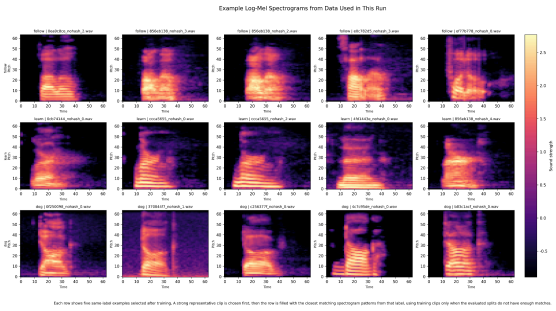
Metric	Value
Preset	medium_end
Validation Accuracy	92.70%
Validation Macro-F1	92.70%
Test Accuracy	91.60%
Test Macro-F1	91.59%
Number of Classes	34
Training Samples	10,000
Validation Samples	1,000
Testing Samples	1,000
Best Threshold	0.80
Validation Expected Cost (Baseline)	0.37
Validation Expected Cost (Tuned)	0.21
Validation Cost Reduction	0.16
Test Expected Cost (Baseline)	0.42
Test Expected Cost (Tuned)	0.26
Test Cost Reduction	0.16

- Final metrics for the medium_end run, with 91.60% test accuracy, 91.59% test Macro-F1, and reduced expected test cost from 0.420 to 0.257 after threshold tuning.

II. Ablation 1 (medium_end with no_augment)

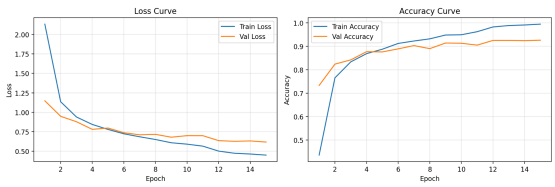
The no_augment ablation removes data augmentation during training to show how the model performs when it learns only from the original audio samples without added noise, shifting, or masking.

Sample Log-Mel Spectrograms (medium_end with no_augment)



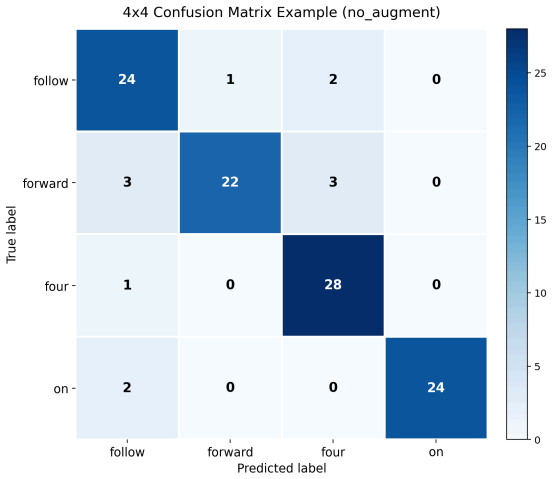
- Side-by-side examples of labeled Log-Mel spectrograms from the audio samples follow, learn, dog.

Training and Validation Accuracy/Loss Curves (medium_end with no_augment)



- Training and validation curves for the no_augment run, showing steady learning across 15 epochs, with decreasing loss and improving accuracy even without data augmentation.

Confusion Matrix (medium_end with no_augment)



- This confusion matrix from the no_augment run shows that most predictions are still correct, but the model sometimes mixes up similar-sounding words like forward, follow, and four. Among the four labels, four and on were recognized more consistently, while forward showed the most confusion.

Final Metrics Summary (medium_end with no_augment)

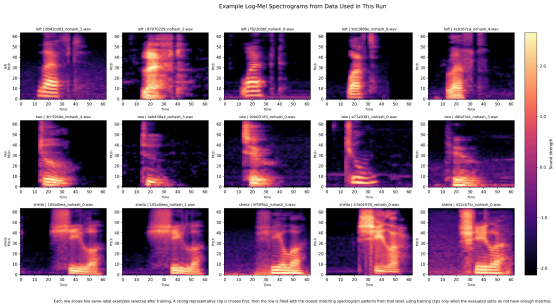
Metric	Value
Preset	medium_end
Ablation	no_augment
Validation Accuracy	92.60%
Validation Macro-F1	92.58%
Test Accuracy	91.70%
Test Macro-F1	91.69%
Number of Classes	34
Training Samples	10,000
Validation Samples	1,000
Testing Samples	1,000
Best Threshold	0.70
Validation Expected Cost (Baseline)	0.37
Validation Expected Cost (Tuned)	0.19
Validation Cost Reduction	0.18
Test Expected Cost (Baseline)	0.42
Test Expected Cost (Tuned)	0.24
Test Cost Reduction	0.17

Final metrics for the no_augment run, showing that the model still performed strongly without data augmentation, with 91.70% test accuracy, 91.69% test Macro-F1, and a lower expected test cost from 0.42 to 0.24 after threshold tuning.

III. Ablation 2 (medium_end + no_balance)

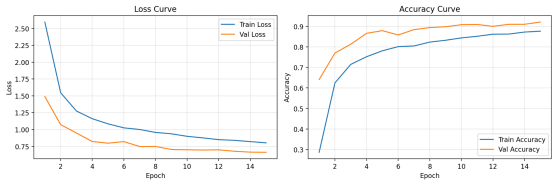
The no_balance run removes the class-balancing step during training to show how the model performs when it learns from the data without extra balancing for class distribution.

Sample Log-Mel Spectrograms (medium_end with no_balance)



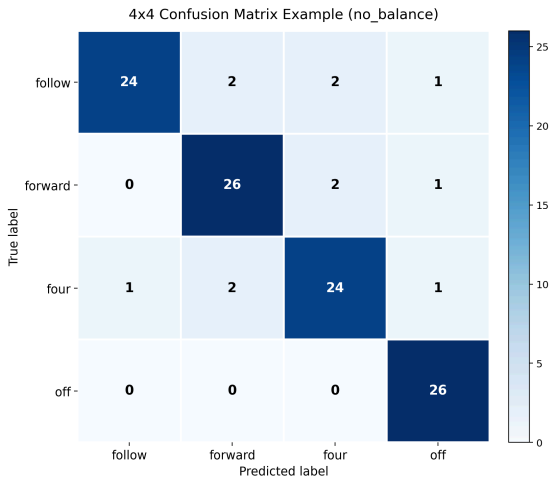
- Example Log-Mel spectrograms from the medium_end run, showing sample clips for the words left, two, and sheila and the variation in their time-frequency patterns across different recordings.

Training and Validation Accuracy/Loss Curves (medium_end with no_balance)



- Training and validation curves for the no_balance run, showing that the model still learned steadily across 15 epochs, with loss going down and accuracy improving even without class balancing.

Confusion Matrix (medium_end with no_balance)



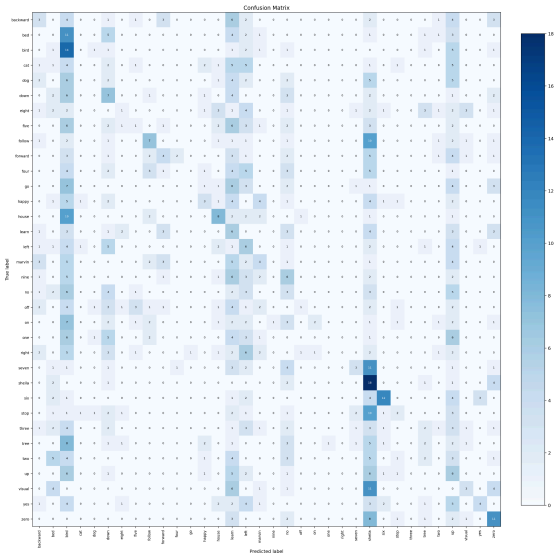
- The confusion matrix from the no_balance run shows that most predictions are still correct, but the model sometimes mixes up similar words like follow, forward, and four, while off was recognized more consistently.

Final Metrics Summary (medium_end with no_balance)

Metric	Value
Preset	medium_end
Ablation	no_balance
Validation Accuracy	92.10%
Validation Macro-F1	92.11%
Test Accuracy	90.90%
Test Macro-F1	90.82%
Number of Classes	34
Training Samples	10,000
Validation Samples	1,000
Testing Samples	1,000
Best Threshold	0.65
Validation Expected Cost (Baseline)	0.40
Validation Expected Cost (Tuned)	0.22
Validation Cost Reduction	0.17
Test Expected Cost (Baseline)	0.46
Test Expected Cost (Tuned)	0.23
Test Cost Reduction	0.22

- Final metrics for the no_balance run, showing 90.90% test accuracy, 90.82% test Macro-F1, and a reduction in expected test cost from 0.46 to 0.23, for a total test cost reduction of 0.22.

Non-DL Baseline Confusion Matrix (medium_end)



- The non-deep-learning baseline produced a much noisier confusion matrix with many off-diagonal errors, showing that a simple centroid-based classifier could not separate the command classes well. This highlights why the CNN was needed for the final system.

The earlier parts of this report show what was first proposed for the project. The next section presents what was actually implemented, the updated experimental setup, and the final results of the completed system.

9. FINAL REPORT

• Problem and Motivation, Success Metrics, and Constraints

This project aims to improve the reliability of speech command recognition in Voice User Interfaces (VUIs), especially under real-world conditions where performance is affected by background noise, accent variation, and differences in speaking style. Project success is measured

using accuracy, Macro-F1, and expected cost after threshold tuning. The model achieved 91.60% test accuracy, 91.59% test Macro-F1, and reduced expected cost from 0.420 to 0.257. The main constraints include background noise, accent variation, differences in speaking style, and limited hardware resources that affect model training.

• Dataset Used (Source, License, Collection Notes), Preprocessing, and Splits

The project uses the Google Speech Commands Dataset v0.02 which contains 105,829 audio recordings consisting of one second and 16kHz spoken commands. The medium_end experiment used a controlled subset of the dataset consisting of 10,000 training samples, 1,000 validation samples, and 1,000 testing samples.

For preprocessing, the audio clips were organized through a dataset manifest. Unusable clips were excluded, and the remaining clips were converted into normalized log-Mel, delta, and delta-delta spectrogram features.

For data splitting, the project follows the dataset's provided validation and testing lists, while the remaining usable clips are assigned to the training set to reduce data leakage.

• Training details: hyperparameters, schedules, compute used, reproducibility steps.

The project used the medium_end preset, which trains on a smaller controlled subset of the Google Speech Commands dataset instead of the full 105,829 audio files. This setup used 10,000 training samples, 1,000 validation samples, and 1,000 testing samples to balance performance, training time, and hardware limitations.

The model was trained for 15 epochs with a batch size of 16, a learning rate of 0.001, a weight decay of 0.0001, and label smoothing of 0.05. To improve robustness, the training pipeline applied time shifting, Gaussian noise, background-noise mixing, gain adjustment, and SpecAugment masking. Balanced sampling and class-weight loss were used to address any class imbalance, while AdamW optimizer with ReduceLROnPlateau learning-rate scheduler was used for training.

The training started with a learning rate of 0.001, which was reduced to 0.0005 in the final epochs, while epoch 14 produced the best checkpoint overall. For compute, the run used automatic device selection, 0 DataLoader workers, feature caching, and AMP disabled for stability. Reproducibility was supported through the fixed seed 2518392709, the saved dataset manifest, the saved run configuration, and the one-command workflow `python .\run.py --preset medium_end --no-open-files`.

• Modeling: architecture(s), why chosen, CNN/NLP/RL components, and how they connect.

The project uses a hybrid architecture composed of a CNN classifier and an RL-based threshold-tuning module. There is no NLP component in this system because the task focuses on speech command classification from audio rather than text processing. Each 1-second audio clip is first converted into normalized log-Mel, delta, and delta-delta spectrogram features. These features are then passed into a residual squeeze-excitation CNN with a convolutional stem, progressively increasing channels, adaptive average pooling, and a classifier head.

The CNN was chosen because spectrograms are two-dimensional time-frequency representations, and CNNs are effective at learning local spatial patterns from this type of input. The RL component was chosen because the project does not only aim to classify commands, but also to improve decision reliability by tuning the confidence threshold in a cost-sensitive way.

After classification, the CNN outputs class probabilities for each command class. These probabilities are then used by a Q-learning RL agent to search for the best confidence threshold on the validation set. In the final decision stage, the selected threshold was applied to reject low-confidence predictions. In this way, the CNN handled feature learning and classification, while the RL component improved decision reliability by adjusting the prediction threshold.

• Evaluation: metrics, baselines, ablations (≥2), error/slice analysis, calibration

To make the comparison fair, we included both a simple non-deep-learning baseline and our CNN-based model. For the non-DL baseline, we used a nearest-centroid classifier on the same log-Mel features, but it performed much worse, reaching only 11.10% validation accuracy and 13.10% test accuracy, with a confusion matrix showing many misclassifications. In contrast, CNN captured the speech patterns much more effectively and achieved 91.60% test accuracy and 91.59% test Macro-F1. This large performance gap shows that a simple classical method was not enough for the speech command classification task, which is why the CNN was chosen as the final model.

In the medium_end run, threshold tuning selected a confidence threshold of 0.80 and reduced the expected test cost from 0.420 to 0.257. Two ablation studies were also conducted. The

no_augment setting achieved 91.70% test accuracy and 91.69% test Macro-F1, with the tuned test cost reduced to 0.241, indicating that further tuning may still improve performance. In contrast, the no_balance setting dropped to 90.90% test accuracy and 90.82% test Macro-F1, suggesting that the balancing strategy used in the main run was helpful overall.

Error analysis was performed using the confusion matrix, which showed common misclassifications between similar commands such as forward and four, down and cat, and no and nine. The project included class-level error analysis through the test-set confusion matrix, but it did not include a separate slice analysis for demographic, speaker, or accent-based subgroups because those metadata were not available in the dataset. No explicit calibration module was added; instead, confidence scores were used directly in the RL-based threshold-tuning stage.

• Ethics & Policy: risks, mitigations, intended use & limitations, privacy, fairness checks.

This project is intended for limited-vocabulary speech command recognition in Voice User Interfaces and similar low-risk applications. It is not designed for surveillance, speaker identification, or high-stakes decision-making. The main risks include misrecognized commands, reduced performance in noisy environments, and unequal performance across accents or speaking styles. To reduce these risks, the project uses augmentation, class-balancing methods, and RL-based thresholding that can reject low-confidence predictions. The project protected privacy by using a public dataset where participant information such as age, gender, and location was not included, and no additional personal data was collected. Formal fairness auditing could not be conducted due to the absence of demographic metadata in the dataset; therefore, fairness evaluation was limited to class-level performance metrics and confusion matrix error analysis. A major limitation of the model is that it was trained exclusively on short English command clips, which may reduce its ability to generalize to other languages or more diverse speech tasks.

• Model Card summary with deployment guidance/disclaimers.

The project's model is a residual squeeze-excitation CNN combined with a Q-learning RL threshold-tuning module for limited-vocabulary speech command recognition.

It uses the Google Speech Commands Dataset v0.02 and processes 1-second, 16 kHz audio clips by converting them into normalized log-Mel, delta, and delta-delta spectrogram features. In the official medium_end run, the model achieved 91.60% test accuracy and 91.59% test Macro-F1, while threshold tuning reduced expected test cost from 0.420 to 0.257. The model is intended for low-risk Voice User Interface tasks such as simple command recognition and interface control. It should not be used for speaker identification, surveillance, or other high-stakes applications.

Deployment should include a fallback or abstain mechanism for low-confidence predictions, validation in the target acoustic environment, and awareness that performance may drop under background noise, accent variation, different speaking styles, or non-English speech.

MODEL CARD

Field	Details
Model	CNN classifier with residual and squeeze-excitation components + RL threshold agent
Dataset	Google Speech Commands v0.02 (Warden, 2018)
License	Creative Commons BY 4.0 license.
Size	105,829 clips, 34 classes, ~2,600 speakers
Splits	Provided validation/testing lists; medium_end used 10,000 / 1,000 / 1,000
Cleaning	252 invalid clips removed (<0.3% of total)
Input	1-sec, 16 kHz audio -> normalized log-Mel, delta, and delta-delta features
Targets	Accuracy ≥90%, Macro-F1 ≥0.88, RL cost reduction ≥20%
Performance	The model achieved 91.60% test accuracy and 91.59% test Macro-F1, while RL

	threshold tuning reduced the expected test cost from 0.420 to 0.257.
Limitation	Performance may drop under noise, accent variation, non-English speech, and hardware limitations during training or deployment.
Intended Use	Academic research and low-risk VUI use; not for high-stakes deployment

• How to run: one-command reproduction, environment setup, and demo instructions.

The project is designed to be reproducible through a simple setup and one-command workflow. The recommended environment uses Python 3.11. The environment can be prepared by creating the Conda environment from environment.yml, activating it, and installing the required packages from requirements.txt. If necessary, a system-specific PyTorch build should be installed first before the remaining dependencies.

For one-command reproduction, the official final configuration uses the medium_end preset. After restoring the dataset and generating the dataset manifest, the full pipeline can be executed with:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\-Programming\Project-root - Copy\6INTELSYS-FINAL-PROJECT-GROUP-10> python .\run.py --preset medium_end --no-open-files
```

```
Choose a run preset:
  1. test - Quick sanity check on a tiny subset. [train=256, val=64, test=64]
  2. low_end - Small run for weaker CPU or RAM limits. [train=1000, val=200, test=200]
  3. medium_end - Balanced default for normal laptops or desktops. [train=8000, val=800, test=800]
  4. high_end - Longest run using the full dataset when available. [train=full, val=full, test=full]
Preset [default: medium_end]:
```

python .\run.py --preset medium_end

This command runs preprocessing, training, and evaluation in sequence. For demonstration, the dataset may first be checked or restored using `cd .\data` and `python .\get_data.py`, then the manifest can be built using:

```
PS C:\-Programming\Project-root - Copy\6INTELSYS-FINAL-PROJECT-GROUP-10> & C:\Users\amiel\AppData\Local\Python\pythoncore
.\pipeline.py"
Manifest written to: C:\-Programming\Project-root - Copy\6INTELSYS-FINAL-PROJECT-GROUP-10\data\dataset_manifest.csv
Total samples: 103706
Split counts: training=83119, validation=9788, testing=10799
Status counts: keep=103454, remove=252
Unmatched list entries: validation=193, testing=209, removed=0
PS C:\-Programming\Project-root - Copy\6INTELSYS-FINAL-PROJECT-GROUP-10>
```

python .\src\data_pipeline.py

The trained model can then be evaluated using:

```
Copy\6INTELSYS-FINAL-PROJECT-GROUP-10> python .\src\eval.py --preset medium_end
```

```
Choose an evaluation preset:
  1. test
  2. low_end
  3. medium_end
  4. high_end
Preset [default: medium_end]: 3
Preset: medium_end
```

```
Epochs completed: 15
Best epoch: 15
Best validation accuracy: 0.925
Final validation accuracy: 0.925
Final validation macro-F1: 0.9249087670821188
Final test accuracy: 0.89875
Final test macro-F1: 0.8980918972660756
Generalization gap: -0.042750000000000066
Batch size: 16
Epoch target: 15
Best tuned threshold: 0.75
Validation expected cost: 0.375 -> 0.1775
Test expected cost: 0.50625 -> 0.2325
Validation relative reduction: 0.5266666666666667
Test relative reduction: 0.5407407407407406
```

python .\src\eval.py --preset medium_end

The final metrics, confusion matrix, and other generated outputs are stored in the experiments/results/medium-end/ directory, while logs and training curves are saved in experiments/logs/medium-end/.

10. TEAM ROLES & MILESTONES

The project team is organized into four key roles to ensure efficient development and successful system integration. Specifically, the team is divided into these specific roles for the project:

- Member 1 - Kean as the Project Lead / Integration oversees the project's overall direction, coordination, and progress, ensuring that data preparation, model development, evaluation, and deployment remain aligned with the objectives and timeline. This role also manages task distribution, team communication, and the integration of outputs into a unified system.
- Member 2 - Glen would be the Data & Ethics Lead manages the dataset and ensures that all data-related processes follow ethical and legal standards. Responsibilities include sourcing and validating datasets, checking data quality and suitability, organizing preprocessing, ensuring compliance with licensing requirements, and addressing fairness, bias mitigation, privacy protection, and responsible AI practices.
- Member 3 - Amiel as the Modeling Lead focuses on designing, building, and improving the machine learning models used in the project. This includes selecting an appropriate CNN architecture for speech classification,

preparing the Mel-spectrogram input pipeline, tuning model parameters, and monitoring training performance to improve accuracy, robustness, efficiency, and generalization.

- Member 4 - Josh as the Evaluation & MLOps Lead handles system testing and the operational side of the machine learning workflow. This role evaluates model performance using metrics such as accuracy, macro-F1 score, and cost reduction, documents experimental results, and compares outcomes against project goals. It also manages experiment tracking, version control, runtime monitoring, reproducibility, and deployment practices to ensure that the system can be maintained and replicated effectively.

To maintain steady progress, the project is divided into structured development milestones.

- Week 1 Proposal & Setup focuses on finalizing the project proposal, defining the workflow, and preparing the required tools, environment, and resources.
- Week 2 Dataset Preparation, Baseline Training, and Initial CNN Results centers on dataset preparation, audio preprocessing, baseline model training, and obtaining initial CNN performance results.
- Week 3 RL Integration, Final Evaluation, and Report Preparation involves integrating the reinforcement learning component, conducting final system evaluation, and completing the project report and documentation.

11. REFERENCES

[1] Pete Warden. 2018. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. arXiv:1804.03209. Retrieved from https://arxiv.org/abs/1804.03209

[2] Sandipan Dhar, Anuvab Sen, Aritra Bandyopadhyay, Nanda Dulal Jana, Arjun Ghosh, and Zahra Sarayloo. 2023. Differential Evolution Algorithm based Hyper-Parameters Selection of Convolutional Neural Network for Speech Command Recognition. arXiv:2310.08914. Retrieved from https://arxiv.org/abs/2310.08914

[3] Dejan Vujičić, Đorđe Damnjanović, Dušan Marković, and Zoran Stamenković. 2025. Deep Learning System for Speech Command Recognition. *Electronics* 14, 19 (Sep. 2025), 3793. https://doi.org/10.3390/electronics14193793

[4] Sidahmed Lachenani, Hamza Kheddar, and Mohamed Ouldzmirli (2025). Improving pretrained YAMNET for enhanced speech command detection via transfer learning. arxiv.2504.19030. Retrieved from https://doi.org/10.48550/arxiv.2504.19030

[5] Jenny Yang, Rasheed El-Bouri, Odhran O'Donoghue, Alexander S. Lachapelle, Andrew A. S. Soltan, and David A. Clifton (2022). Deep Reinforcement Learning for Multi-class Imbalanced Training. arXiv:2205.12070. Retrieved from arXiv:2205.12070

[6] Kevin Wilkinghoff, Alessia Cornaggia-Urrigshardt, and Zheng-Hua Tan. (2025). Quantization-Based Score Calibration for Few-Shot Keyword Spotting with Dynamic Time Warping in Noisy Environments. arXiv.2510.15432 Retrieved from https://doi.org/10.48550/arXiv.2510.15432